

A green, glowing padlock is positioned in the center-left of the image. It is set against a dark background with a complex, glowing circuit board pattern. The padlock is slightly transparent, revealing a blue padlock underneath it. The overall aesthetic is high-tech and digital.

# Applied Cryptography

Week 4 7 Bit PDU Encryption

# 7 bit PDU Encryption



7 bit PDU is a compression method utilised by some mobile phones to reduce SMS sizes.



It's still found today (although less common) and it is an important tool in the mobile forensic investigator's toolkit



Characters on SMS messages are limited to 140 bytes for the actual message + additional bytes used for routing etc



If you can compress the message size you can



Send more data per message



Reduce network traffic



# 7 bit PDU

- We will concentrate solely on how the message is compressed and ignore the 'data management' information



# Ascii Encoding

Using the ASCII encoding each character can be represented by exactly one byte (8 bits) which give 256 possible values.

SMS messages only use the first 128 of these (0 - 127) which mean that the most significant bit is always 0

00000000 0 0  01111111 127 7F

<https://www.asciitable.com/>

# 7bit PDU Encryption

1

7bit encryption works by getting rid of all those 0s to reduce the byte size down from 8 bytes to 7

2

This way for every 140 characters we originally had we can now send another 20

3

Unfortunately, it's not quite that simple, we need to restructure the bytes so that we still send and receive in blocks of 8 bits.

# 7bit PDU An Example

- Take the 10 byte message HelloHello

|       |   |   |   |   |   |   |   |   |   |    |
|-------|---|---|---|---|---|---|---|---|---|----|
| Bytes | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| Text  | H | e | l | l | o | H | e | l | l | o  |

- The first stage is to find the ASCII values for the characters, I'm using Hex

|                     |          |          |          |          |          |          |          |          |          |          |
|---------------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| Bytes               | 1        | 2        | 3        | 4        | 5        | 6        | 7        | 8        | 9        | 10       |
| Text                | H        | e        | l        | l        | o        | H        | e        | l        | l        | o        |
| ASCII <sub>16</sub> | 48       | 65       | 6C       | 6C       | 6F       | 48       | 65       | 6C       | 6C       | 6F       |
| Binary              | 01001000 | 01100101 | 01101100 | 01101100 | 01101111 | 01001000 | 01100101 | 01101100 | 01101100 | 01101111 |

# 7bit PDU

- Next we drop all those leading zeroes to create 7 bit binary strings

| Bytes               | 1       | 2       | 3       | 4       | 5       | 6       | 7       | 8       | 9       | 10      |
|---------------------|---------|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| Text                | H       | e       | l       | l       | o       | H       | e       | l       | l       | o       |
| ASCII <sub>16</sub> | 48      | 65      | 6C      | 6C      | 6F      | 48      | 65      | 6C      | 6C      | 6F      |
| 7bit                | 1001000 | 1100101 | 1101100 | 1101100 | 1101111 | 1001000 | 1100101 | 1101100 | 1101100 | 1101111 |

- Now we're ready to perform the algorithm

# Substituting bits

- Take the least significant bit of the 2<sup>nd</sup> byte and add it to the start of the 1<sup>st</sup> byte

| Bytes               | 1                | 2               |
|---------------------|------------------|-----------------|
| Text                | H                | e               |
| ASCII <sub>16</sub> | 48               | 65              |
| 7bit                | 1001000          | 110010 <b>1</b> |
| Adjusted Bytes      | <b>1</b> 1001000 |                 |

Byte one is now the required 8 bytes but byte 2 is only 6 bytes long. We're going to take 2 bits from byte 3 and prepped them to byte 2

| Bytes               | 1        | 2                | 3               |
|---------------------|----------|------------------|-----------------|
| Text                | H        | e                | l               |
| ASCII <sub>16</sub> | 48       | 65               | 6C              |
| 7bit                | 1001000  | 110010           | 11011 <b>00</b> |
| Adjusted Bytes      | 11001000 | <b>00</b> 110010 |                 |



# Substituting bits

- The process continues taking 3 bits from byte 4 and adding them to byte 3

| Bytes               | 1        | 2        | 3        | 4       |
|---------------------|----------|----------|----------|---------|
| Text                | H        | e        | l        | l       |
| ASCII <sub>16</sub> | 48       | 65       | 6C       | 6C      |
| 7bit                | 1001000  | 1100101  | 1101100  | 1101100 |
| Adjusted Bytes      | 11001000 | 00110010 | 10011011 |         |

# Substituting Bits

- If we roll this forward to the 8<sup>th</sup> byte, I now prepend all of the bits. This is where I gain a byte. As I can start again with byte 9 prepending 1 bit from byte 10. The compression means I only need 8 bytes to send 9 bytes of data.

| Bytes               | 1        | 2        | 3        | 4        | 5        | 6        | 7        | 8       | 9       | 10      |
|---------------------|----------|----------|----------|----------|----------|----------|----------|---------|---------|---------|
| Text                | H        | e        | l        | l        | o        | H        | e        | l       | l       | o       |
| ASCII <sub>16</sub> | 48       | 65       | 6C       | 6C       | 6F       | 48       | 65       | 6C      | 6C      | 6F      |
| 7bit                | 1001000  | 1100101  | 1101100  | 1101100  | 1101111  | 1001000  | 1100101  | 1101100 | 1101100 | 1101111 |
| Adjusted Bytes      | 11001000 | 00110010 | 10011011 | 11111101 | 01000110 | 10010110 | 11011001 | 1101100 |         |         |
| Bits Taken          |          | 1        | 2        | 3        | 4        | 5        | 6        | 7       |         |         |

# Padding the final byte

- This just leaves the final byte which in this example only has 6 bits
- The solution is to pad the final byte with zeroes until it is 8 bits long

| 8       | 9       | 10     |
|---------|---------|--------|
|         |         | o      |
| 6C      | 6C      | 6F     |
| 1101100 | 1101100 | 110111 |
| 1110110 |         |        |
| 0       |         |        |
| 7       |         | 1      |

| 8        | 9        | 10 |
|----------|----------|----|
|          |          | o  |
| 6C       | 6C       | 6F |
| 1101100  | 1101100  |    |
| 11101100 | 00110111 |    |
| 7        |          |    |

# Completed algorithm

- The completed algorithm is below note we have reduced the 10 byte input to a 9 byte output whilst still representing all the data.

|                     |          |          |          |          |          |          |          |          |          |         |
|---------------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|---------|
| Bytes               | 1        | 2        | 3        | 4        | 5        | 6        | 7        | 8        | 9        | 10      |
| Text                | H        | e        | l        | l        | o        | H        | e        | l        | l        | o       |
| ASCII <sub>16</sub> | 48       | 65       | 6C       | 6C       | 6F       | 48       | 65       | 6C       | 6C       | 6F      |
| 7bit                | 1001000  | 1100101  | 1101100  | 1101100  | 1101111  | 1001000  | 1100101  | 1101100  | 1101100  | 1101111 |
| Adjusted Bytes      | 11001000 | 00110010 | 10011011 | 11111101 | 01000110 | 10010110 | 11011001 | 11101100 | 00110111 |         |
| ASCII <sub>16</sub> | C8       | 32       | 9B       | FD       | 46       | 96       | D9       | EC       | 37       |         |

# Decryption

- Decryption is simply the inverse process
- For each byte take the borrowed bits generated by the encryption and append them to the next byte
- Lastly pad each newly created 7bit pattern with a 0 to restore the 8 bit octets
- This blog post gives a nice overview of the whole process
- <https://doubleblak.com/blogPosts.php?id=6>